

نسخه ۱.۱ (MVP)

تیم: Frontend / Backend / AI

افق زمانی: نقشه‌ی راه فشرده‌ی ۴ ماهه (در بودجه‌ی کلی ۶ ماه تا ۱ سال)، بودجه محدود

تاریخ ویرایش سند: ۱۴۰۵/۰۵/۱۰

۱. چشم‌انداز و مسئله

مسئله: شرکت‌های تولیدکننده‌ی برد الکترونیکی (PCB/PCBA) در ایران، برای تامین قطعات یک BOM، مجبورند دستی بین چند فروشنده بگردند، قیمت و موجودی را مقایسه کنند و بهترین ترکیب خرید را پیدا کنند، فرآیندی زمان‌بر، وابسته به تجربه‌ی فردی و بدون دید جامع.

راه حل MVP: کاربر یک فایل BOM (اکسل) آپلود می‌کند، سیستم قیمت و موجودی لحظه‌ای را از ۱۰ فروشنده‌ی ازپیش‌منتخب جمع‌آوری می‌کند، قطعات را با کاتالوگ داخلی تطبیق می‌دهد و بهترین پیشنهاد خرید (بر اساس قیمت، موجودی، تعداد فروشنده و ...) را برمی‌گرداند، با بازبینی یک اپراتور در فاز اول.

معیار موفقیت ۴ ماهه: سیستم به‌طور پایدار یک BOM واقعی را می‌گیرد و بهترین سبد خرید بین ۱۰ فروشنده را با قیمت/موجودی لحظه‌ای برمی‌گرداند.

دارایی استراتژیک بلندمدت: با تجمیع BOMهای واقعی، تصمیمات کاربران و داده‌ی فروشندگان در طول زمان، سیستم به یک Knowledge Graph قطعات/جایگزین‌ها/فروشنندگان تبدیل می‌شود که کپی‌کردنش برای رقبا سخت است، اما ساخت این گراف به بعد از اثبات MVP موکول می‌شود.

۲. اصول طراحی (چرا این معماری)

۱. ساده‌ترین چیزی که کار می‌کند، نه کامل‌ترین چیزی که ممکن است. با ۳ نفر و بودجه‌ی محدود، سرعت رسیدن به یک محصول قابل تست از هر بهینه‌سازی فنی مهم‌تر است.

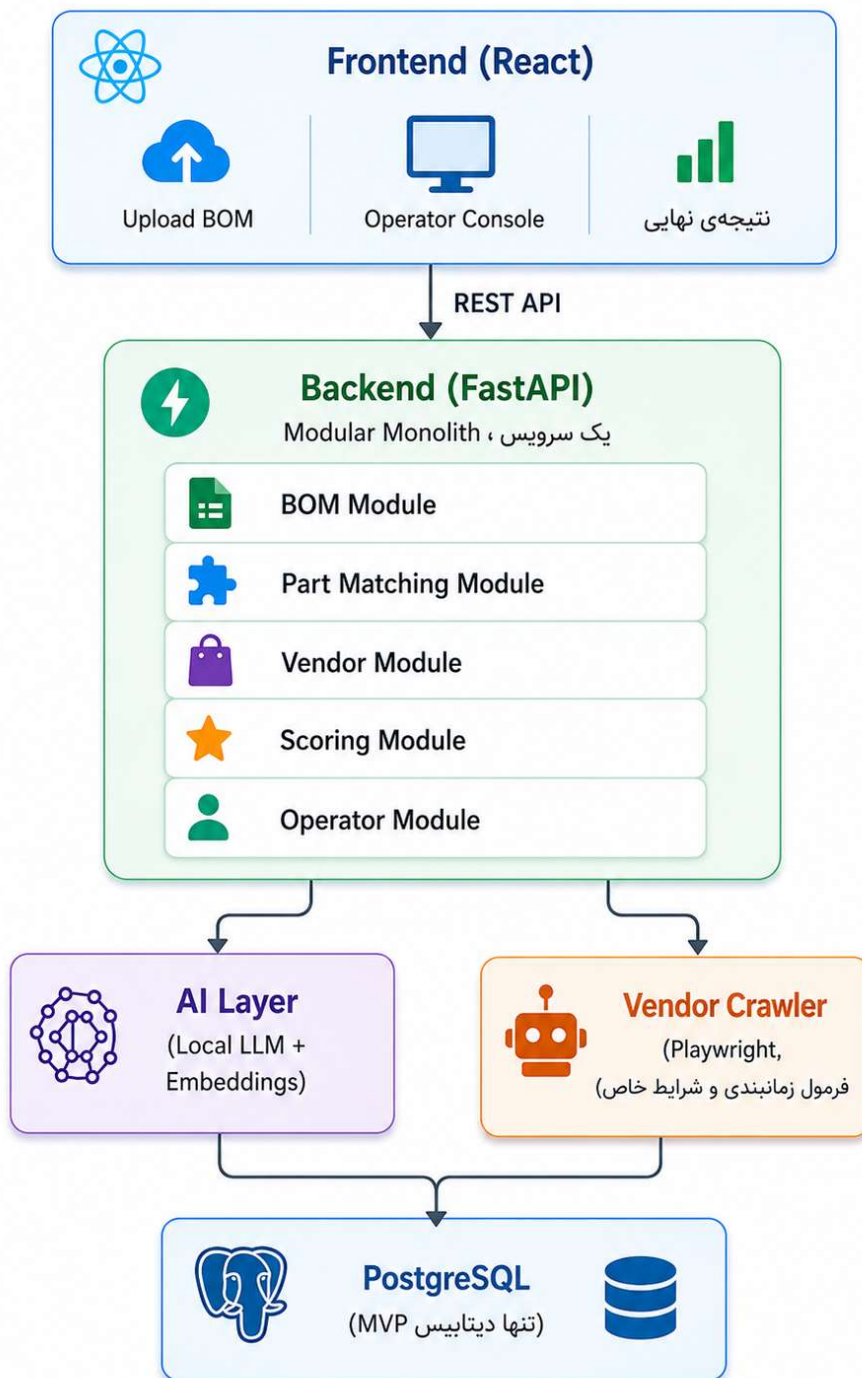
۲. تست ریسک‌دارترین فرض در ابتدای کار. فرض پریسک این پروژه "آیا می‌شود ۱۰ سایت فروشنده را به‌طور پایدار Crawl کرد" خواهد بود، نه اینکه "آیا AI می‌تواند BOM را پارس کند". Crawler از هفته‌ی اول، موازی با بقیه‌ی کارها ساخته و تست می‌شود.

۳. محرمانگی BOM مشتری = پردازش محلی. برای رسیدن سریع‌تر به MVP، پردازش AI در فاز اول (ماه ۱ تا ۳) از طریق API مدل‌های ابری رایگان/ارزان انجام می‌شود؛ این یعنی محتوای BOM در این فاز به یک سرویس شخص ثالث ارسال می‌شود و باید صریحاً به مصرف‌کننده‌های بالقوه اعلام شود. بعد از اثبات MVP (ماه ۴)، پردازش به سمت LLM محلی (self-hosted، روی GPU محلی) منتقل می‌شود تا این وابستگی و ریسک محرمانگی برطرف شود.

۴. اپراتور از ابتدا در حلقه‌ی اول قرار دارد. در فاز اول همه‌ی خروجی نهایی توسط یک اپراتور بازبینی می‌شود؛ AI فقط پیشنهاد می‌دهد. برای ابهام در سطح تک‌ردیف BOM (اطمینان ۷۰٪ تا ۹۵٪)، خود مشتری در پیش‌نمایش فرانت‌اند تایید می‌کند؛ بازبینی نهایی خروجی همچنان بر عهده‌ی اپراتور است.

۵. زیرساخت داده به صورت ساعتی Cache می‌شود، Live Crawl نمی‌شود. قیمت/موجودی فروشنده‌ها به‌صورت دوره‌ای (نه در لحظه‌ی درخواست کاربر) جمع‌آوری و در دیتابیس Cache می‌شود؛ این کش معتبر است تا زیر ۲ ساعت یا تا رخداد جهش نرخ دلار، هرکدام زودتر برسد.

۳. معماری کلی (High-Level Architecture)



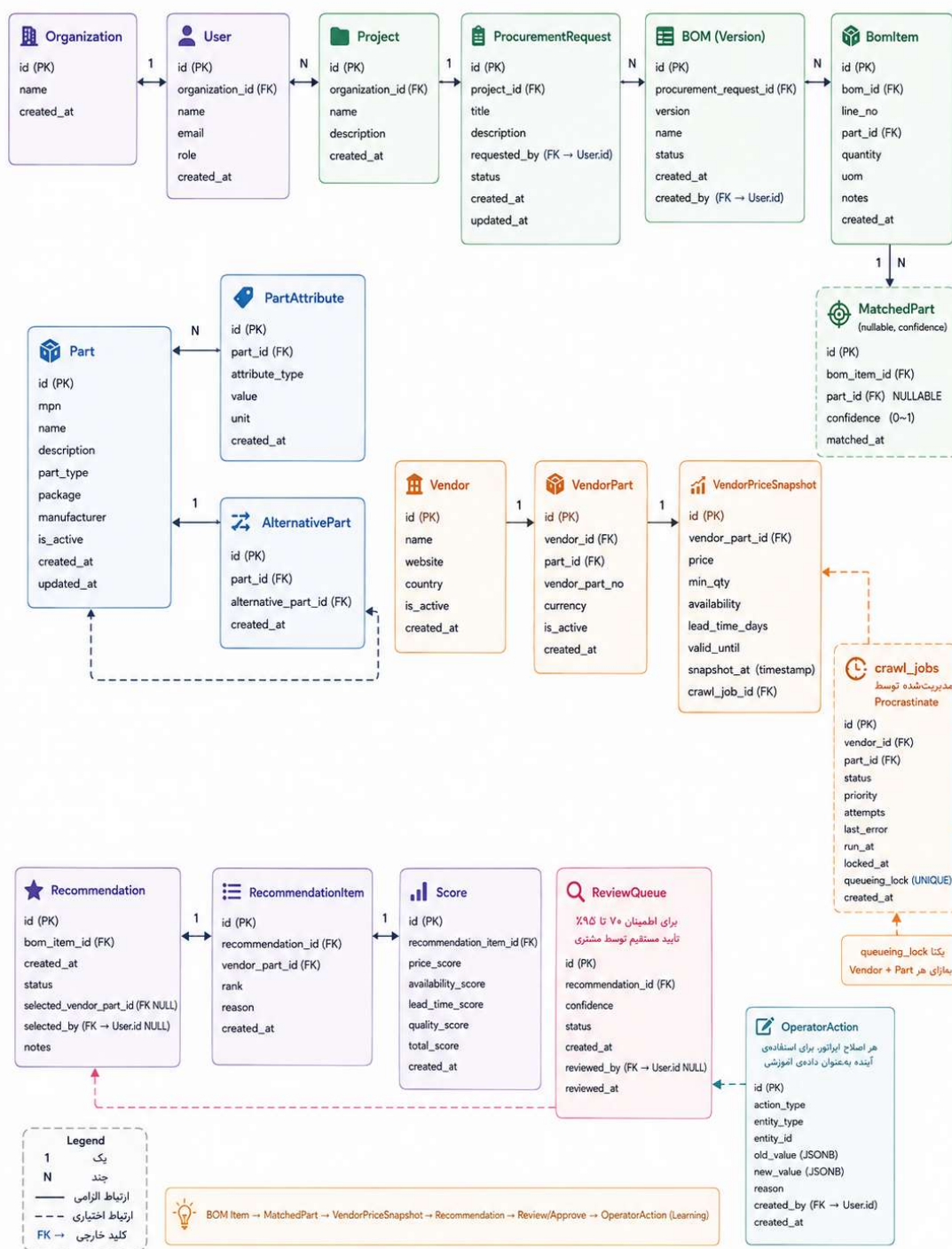
بدون Microservice، بدون Neo4j، بدون Message Queue سنگین (Kafka/RabbitMQ)، این‌ها همه بعد از اثبات کارکرد MVP و در صورت نیاز واقعی اضافه می‌شوند.

۴. استک فنی

لایه	انتخاب	دلیل
Backend	Python + FastAPI	زبان مشترک با AI Layer و Crawler، هماهنگی راحت‌تر
Frontend	React (Vite) + Tailwind	سریع‌ترین مسیر توسعه برای یک توسعه‌دهنده فرانت‌اند
دیتابیس	PostgreSQL	تنها دیتابیس MVP؛ حتی روابط جایگزین قطعات نیز در همین پایگاه داده نگهداری می‌شود
Crawler	Playwright (Python)	صفحات فروشگاه‌های معمولاً JavaScript-heavy هستند، نسبت به Requests و BeautifulSoup قابل‌اعتمادتر است
صف Crawler	Procrastinate (روی PostgreSQL)	صف پایدار روی PostgreSQL با Retry/Backoff و بازبازی خودکار Job گیرکرده، بدون افزودن Redis/RabbitMQ
AI فاز اول	مدل‌های ابری رایگان/ارزان (مثل DeepSeek)	بدون وابستگی به GPU، راه‌اندازی سریع برای شروع در بازه‌ی ۴ ماهه، پردازش واقعی BOM در فاز اول
AI فاز دوم (Fallback)	Ollama + مدل کوانتیزه ۸-۷B (Qwen۲,۵-) روی GPU محلی (مثلاً ۷B یا ۸B-۳,۱- Llama) (RTX ۳۰۶۰ ۸GB)	بعد از اثبات MVP، برای کاهش هزینه‌ی تکرار شونده‌ی API و رفع نگرانی محرمانگی BOM
Embedding سبک (اختیاری)	یک مدل Embedding سبک روی CPU	برای تطبیق سریع نام‌های واضح قطعات قبل از ارجاع موارد مبهم به AI
Deploy	VPS اختصاصی Docker + (Ubuntu ۲۴) Compose	استقرار ساده روی یک VPS اختصاصی، بدون نیاز به Backend:Kubernetes، Crawler و DB همه روی همین VPS

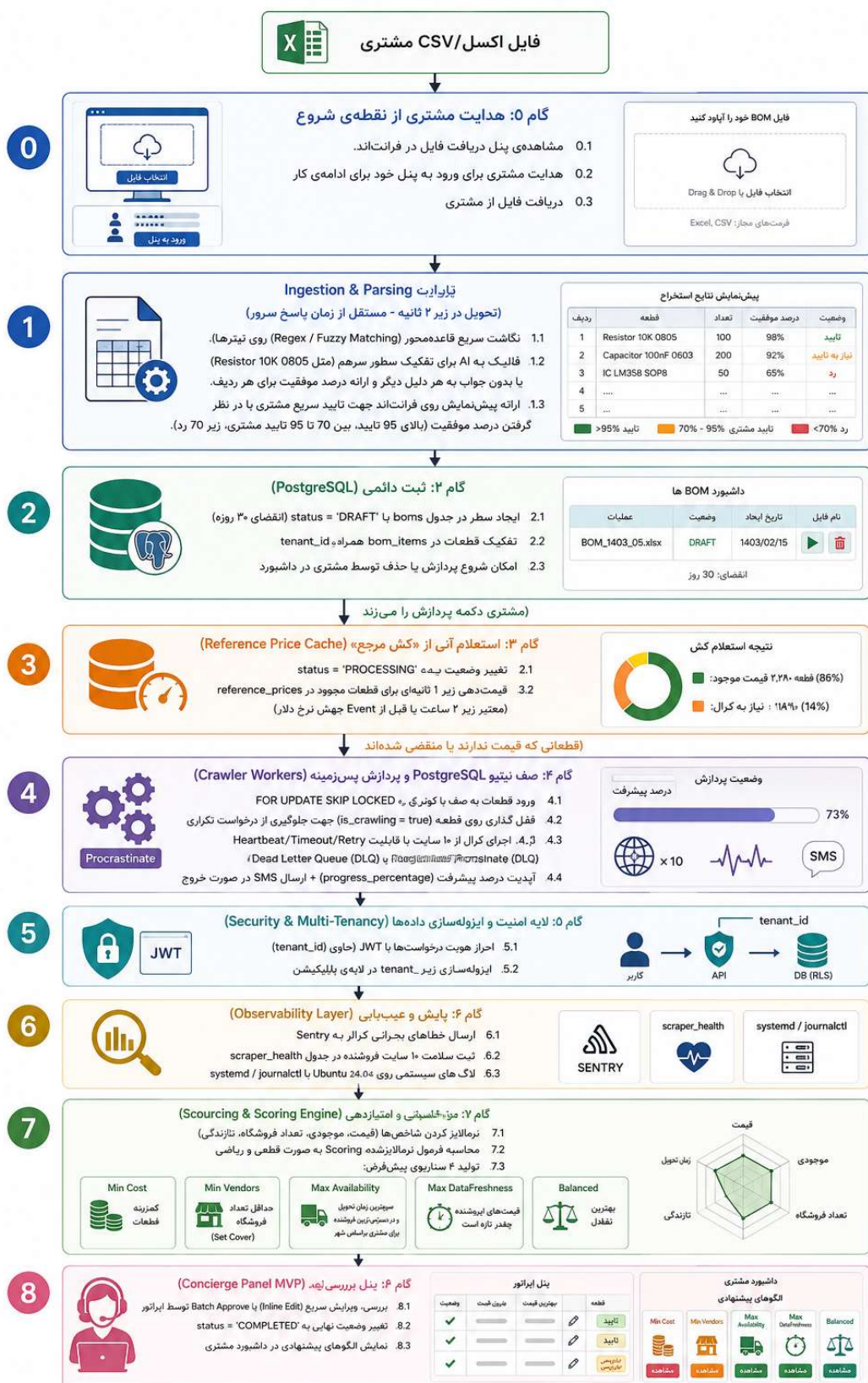
۵. مدل داده (نسخه‌ی فشرده‌ی MVP)

در این مرحله فقط موجودیت‌هایی که واقعاً برای MVP لازمند آورده می‌شوند، Vendor Intelligence، Knowledge Graph، پیشرفته و جداول یادگیری به فاز بعد موکول می‌شوند (به بخش ۹ نگاه کنید).



نکته‌ی مهم: "OperatorAction" را از روز اول ذخیره می‌کنیم، حتی اگر فعلاً استفاده‌ای از آن نکنیم؛ این دقیقاً همان دیتاستی است که بعداً ارزش واقعی خواهد داشت.

۶. پایپ‌لاین اصلی (Core Workflow)



۷. موتور Crawler فروشنده‌ها

- لیست ثابت ۱۰ فروشنده، انتخاب‌شده توسط تیم (فقط سایت‌های فعال و شناخته‌شده).
- هر فروشنده یک Crawler مجزا دارد (چون ساختار HTML هر سایت متفاوت است) — نگهداری این ۱۰ اسکریپت مسئولیت مشخصی است که باید به یک نفر در تیم واگذار شود.
- اجرای دوره‌ای و مشروط: موتور یا مازول Crawler با شرایط خاصی اجرا می‌شود. به صورت پیش‌فرض سیستم دارای یک لیست قیمت پایه می‌باشد (referencePriceList). این لیست مرجع قیمت‌گیری سیستم می‌باشد و در شرایطی که در ادامه توضیح داده می‌شود توسط Crawler به‌روز می‌شود.
- با ورود هر لیست قطعه به صف، سیستم هر قطعه را از لیست قیمت مرجع بررسی می‌کند، در صورتی که این قیمت (یک قطعه خاص از یک فروشنده‌ی خاص) کمتر از دو ساعت پیش به‌روز شده باشد و هشدار نوسان قیمت دلار دریافت نکرده باشد، همین قیمت را ملاک قرار می‌دهد، در غیر این صورت ابتدا قیمت را در لیست مرجع به‌روز کرده سپس به درخواست کننده بر می‌گرداند. این فرمول در فازهای بعدی می‌تواند پیچیده‌تر شود.
- نکته: سایت‌ها ساختارشان را تغییر می‌دهند و ممکن است ربات را مسدود کنند. باید یک مکانیزم مانیتورینگ ساده باشد که وقتی یک Scraper شکست می‌خورد (مثلاً پیوسته صفر نتیجه برمی‌گرداند)، به تیم اطلاع بدهد.
- این بخش، اولین چیزی است که باید در هفته‌ی اول پروژه ساخته و تست شود.

۸. Scoring Engine (نسخه‌ی اول)

فرمول ساده و قابل تنظیم (هر وزن یک عدد config است، نه هاردکد):

FinalScore =

$w_1 \times \text{PriceScore}$

+ $w_2 \times \text{AvailabilityScore}$

+ $w_3 \times \text{VendorCountScore}$ (کمینه کردن تعداد فروشنده در سبد)

+ $w_4 \times \text{DataFreshnessScore}$ (این فروشنده Crawl چقدر تازه است آخرین)

این فرمول پایه برای تولید چند سناریوی خروجی با وزن‌های متفاوت استفاده می‌شود: Min Cost, Min Vendors, Max Availability (بر پایه‌ی یک فیلد "شهر" در دیتاست فروشنده)، و Balanced. AvailabilityScore در نسخه‌ی MVP بر اساس تطابق شهر فروشنده با شهر مشتری محاسبه می‌شود، نه داده‌ی واقعی زمان تحویل.

"DataFreshnessScore" می‌بایست از ابتدا در فرمول وجود داشته باشد، چون قیمت در بازار به سرعت تغییر می‌کند، پیشنهادی که بر اساس دیتای ۲ روز پیش باشد می‌تواند اعتماد مشتری را از بین ببرد.

"VendorTrustScore" (اعتبار فروشنده) و معیارهای پیشرفته‌تر (که در گفتگوی اولیه مطرح شده بودند) به فاز بعد از جمع‌آوری داده‌ی کافی موکول می‌شوند.

۹. آنچه از MVP خارج است

مورد	دلیل
Knowledge Graph / Neo4j	بدون داده کافی، سرمایه‌گذاری زودهنگام است؛ روابط ساده جایگزینی قطعات (Alternative Part) در PostgreSQL کفایت می‌کند.
Fine-tuning مدل اختصاصی	نیازمند دیتاست بزرگ، تمیز و برچسب‌گذاری شده است که در حال حاضر وجود ندارد.
API / همکاری رسمی با فروشندگان	در فاز اول تصمیم بر استفاده از Crawl گرفته شده و نیازی به همکاری مستقیم فروشندگان نیست.
Microservices / Kafka / CQRS	برای مقیاس MVP، پیچیدگی و سربار مهندسی بالایی ایجاد می‌کند بدون اینکه ارزش متناسبی تولید کند.
اتوماسیون کامل (Level ۳-۴)	فاز اول روی Level ۲ متمرکز است؛ یعنی ارائه پیشنهاد و رتبه‌بندی با تأیید نهایی اپراتور.
مدل درآمدی نهایی	پس از گفتگو و اعتبارسنجی با مشتریان بالقوه، مدل درآمدی نهایی تعیین خواهد شد.

۱۰. نقشه‌ی راه

سه مسیر موازی (نه پشت‌سرهم). فشرده‌سازی از ۶ به ۴ ماه یعنی حاشیه‌ی اطمینان کمتری داریم؛ برای همین نقطه‌ی تصمیم‌گیری پرریسک‌ترین بخش (Crawler) به‌جای پایان ماه ۱، همان پایان هفته‌ی چهارم مشخص می‌شود.

مسیر A – Crawler (هفته ۱ تا پایان ماه ۱)

- هفته ۱: انتخاب ۱۰ فروشنده، ساخت Scraper برای ۲-۳ تایی اول
- هفته ۲: تست پایداری اولیه
- هفته ۳-۴: تکمیل بقیه‌ی Scraper ها + راه‌اندازی صف Procrastinate روی PostgreSQL + مانیتورینگ ساده
- **نقطه‌ی تصمیم (پایان هفته ۴):** اگر حداقل ۷-۸ فروشنده به‌طور پایدار Crawl می‌شوند؛ ادامه؛ در غیر این صورت، لیست فروشندگان یا روش Crawl بازبینی شود.

مسیر B – BOM Pipeline (هفته ۱ تا پایان ماه ۲)

- هفته ۱-۲: جمع‌آوری اکسل‌های واقعی از همکاران/آشنایان + طراحی مدل داده + ساخت Parser
- هفته ۲-۳: اتصال به API مدل‌های ابری رایگان/ارزان (AI فاز اول) + Embedding Matching سریع
- هفته ۳-۵: منطق Confidence Threshold (شامل تایید مشتری برای بازه‌ی ۷۰-۹۵٪) + Operator Console پایه
- هفته ۶-۸: تست روی اکسل‌های جمع‌آوری‌شده، با اصلاح دستی اپراتور روی همه‌ی خروجی‌ها

مسیر C – اعتبارسنجی بازار (هفته ۱ تا پایان ماه ۲، بدون فروش)

- گفتگوی کوتاه با ۵-۶ مصرف‌کننده‌ی بالقوه (نه فروش، فقط شنیدن فرآیند فعلی‌شان و نشان دادن نمونه‌ی خروجی)؛ صریحاً اعلام می‌شود که در فاز اول از یک API ابری برای پردازش استفاده می‌شود
- هدف: مطمئن شدن که فرمت خروجی و فرضیات محصول با نیاز واقعی همخوانی دارد، قبل از صرف زمان روی ساخت

تلاقی مسیرها (ماه ۲ پایانی تا ماه ۳)

- اتصال Crawler (مسیر A) به BOM Pipeline (مسیر B): Scoring Engine واقعی روی دیتای واقعی Crawl شده
- تحویل اولین BOM واقعی end-to-end (با بازبینی کامل اپراتور) + خروجی Excel و اعلان ایمیلی

ماه ۴

- تکرار روی فیدبک مصرف‌کننده‌های بالقوه (مسیر C)
- شروع پیاده‌سازی LLM محلی به‌عنوان Fallback (AI فاز دوم) — برای کاهش هزینه‌ی تکرارشوندگی API و رفع نگرانی محرمانگی BOM
- کاهش تدریجی وابستگی به بازبینی دستی اپراتور برای موارد اطمینان بالا
- شروع مکالمات جدی‌تر با مشتریان بالقوه برای اعتبارسنجی مدل درآمدی

۱۱. ریسک‌های اصلی و نحوه‌ی مدیریت

ریسک	نوع	مدیریت
Crawler فروشنده‌ها ناپایدار یا مسدود می‌شود	فنی (بالا)	تست و اعتبارسنجی Crawler از هفته اول توسعه (مسیر A) و پیاده‌سازی مانیتورینگ ساده برای تشخیص شکست Scraper ها
مشتری واقعی به این خروجی نیاز ندارد	بازار (بالا)	انجام مصاحبه و گفتگو با مشتریان بالقوه بدون تلاش برای فروش (مسیر C) پیش از تکمیل محصول
GPU محلی برای حجم بالاتر کافی نیست	فنی (متوسط)	امکان ارتقا به GPU قوی‌تر یا استفاده از سرویس‌های اجاره‌ای در آینده؛ برای MVP کافی است
رقیب نزدیک (مانند Partjoo) وارد حوزه BOM Optimization شود	رقابتی (متوسط)	تمرکز بر مزیت رقابتی اصلی؛ یعنی بهینه‌سازی سبد کامل BOM به جای جستجوی تک‌قطعه و همچنین سرعت بیشتر در ورود به بازار
داده‌های قیمت بیش از حد قدیمی شوند	فنی / اعتماد (متوسط)	در نظر گرفتن Data Freshness Score در فرمول Scoring از همان روز اول برای کاهش اثر داده‌های قدیمی بر نتایج نهایی
حجم دیسک VPS با رشد referencePriceList پر می‌شود	فنی (متوسط)	سیاست نگهداری Archive/برای referencePriceList از هفته‌ی اول، قابل ارتقای فضای VPS در صورت نیاز

۱۲. آنچه هنوز باز است (برای فازهای بعدی)

- مدل درآمدی نهایی: بعد از گفتگو با مصرف‌کننده‌های بالقوه (مسیر C) مشخص می‌شود.
- استراتژی رابطه با فروشنده در بلندمدت (آیا در نهایت به آن‌ها API می‌دهیم؟): بعد از اثبات ارزش محصول.
- زمان دقیق شروع طراحی Knowledge Graph: بعد از جمع‌آوری چند صد BOM واقعی.